Verifier-Anchored RL: A Learned Controller Orchestrating a Frozen LLM over a Structured Reasoning State

Alexander Towell

2026-06-30

0. What this document is

This is the self-contained design of the system: a small external controller network that orchestrates a frozen large language model (LLM) by issuing typed operations over a highly structured, nested reasoning state, with a symbolic verifier as the only source of reward. Section 2 develops the small amount of reinforcement-learning (RL) theory the rest of the document relies on, so no outside reading is needed.

The framing discipline is RL first, search second. The central object is an agent (a policy and a Q function) learning over an induced context Markov decision process (MDP) whose only reward source is a symbolic verifier. Monte Carlo Tree Search (MCTS) is a sample-efficient policy-improvement operator and a test-time compute knob (Sections 7.4 and 8), not the central object. The core training story stands on plain RL.

---

1. Thesis, and what this is not

1.1 The thesis in one paragraph

We train a small external controller network (on the order of a few million parameters: a verb head, a pointer head, and a Q head) that orchestrates a frozen LLM. The controller never edits the LLM; the LLM is a frozen forward-pass component that generates content and, optionally, encodes state. Each action is a pair `(verb, target)`, a lens: it projects a small, focused slice of the structured state into a prompting template, lets the frozen LLM fill in content under a constrained schema, and writes the result back into the neighborhood of one node. The state is not a flat growing transcript of tokens; it is a structured, nested reasoning forest of goals, subgoals, hypotheses, and verified facts. A symbolic verifier (SymPy on a canonicalized answer, a unit-test runner, a proof checker) is the only thing that produces reward.

1.2 The contrast that motivates this document

A reader can mistake this for an o1-style or DeepSeek-R1-style reasoning model. It is not, and the difference is the point. o1 and R1 apply RL to the LLM's own weights so it emits a better chain of thought, token by token. Here the LLM is frozen and the RL operates over a tiny external action space of typed operations on a symbolic state.

Axis	o1 / DeepSeek-R1	This system
What RL optimizes	the LLM's own weights	a small external controller; the LLM is frozen
Action granularity	the next token of a free-form chain of thought	one typed `(verb, target)` operation
State	a flat, growing context of tokens	a structured, nested reasoning forest
What the LLM sees per step	the entire running trace	a lens: one node's grounded-local slice
Reward source	learned reward model, preference, or verifiable reward over the trace	a symbolic verifier only: terminal verdict on the committed answer (plus optional gated `VERIFY` shaping, Section 6)
Cost of "thinking" at deploy	a large model emitting many tokens	a frozen small generator plus a tiny controller; optional search

"Prompt an LLM to reason step by step" is exactly what this system does not do. The LLM is a content-generation oracle that an external policy steers over a symbolic state, and correctness is decided by a program, not by a model's opinion.

1.3 Why the structure earns its place

A small model reasons poorly across a long, flat context but well over a short, on-topic one. The lens exploits this: when the controller acts on a subgoal, the LLM sees that subgoal's small neighborhood, not the entire history. The nested state is the substrate that makes such focusing possible, and the verifier is what keeps the learning signal honest.

---

2. Reinforcement-learning foundations

This section develops the minimum RL the rest of the document uses; its notation recurs in Sections 4 to 7. A reader fluent in MDPs and Q-learning can skim it.

MDP. A discounted MDP is (S, A, P, R, γ): states S; actions $A(s)$ available in state s; transition probability $P(s' \mid s, a)$; immediate reward $R(s, a, s')$; discount $\gamma \in [0, 1)$. The Markov assumption is that the next state depends only on the current state and action, not the full history. Here s is the reasoning forest (Section 3), a is a `(verb, target)` pair (Section 4), P is induced by the frozen LLM, and R comes from the verifier (Section 6). The state space here is discrete but far too large to enumerate or tabulate; the expanded sums below need only discreteness.

Bellman optimality. The optimal action value satisfies, written first as an explicit sum over next states:

$$Q^\star(s, a) \;=\; \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma \max_{a'} Q^\star(s', a') \right].$$

Take a in s, average over next states using P, collect the immediate reward, add the discounted best continuation. The greedy policy is $\arg\max_a Q^\star(s,a)$; a stochastic one is $\mathrm{softmax}_a(Q(s,a)/\tau)$ at temperature τ.

A hand-computable instance, introduced here so the sum has something concrete to bind to: four states, a start S and terminals G, A, B. From S there are two actions. Action *safe* moves to G with probability 1 and reward 4. Action *risky* moves to A with probability 0.8 and reward $+10$, or to B with probability 0.2 and reward -10. Terminal states have value 0, so the explicit sum gives $Q^\star(S,\text{safe}) = 1.0\,(4+\gamma\cdot 0) = 4$ and $Q^\star(S,\text{risky}) = 0.8\,(10+\gamma\cdot 0) + 0.2\,(-10+\gamma\cdot 0) = 6$: so $V^\star(S) = 6$ and the greedy policy takes the risky action. The value of an action is its probability-weighted average outcome, not its best or worst case. This tiny MDP doubles as ground truth for the classical validation anchors of Section 10.

Q-learning from samples. P cannot be enumerated, since it integrates over the LLM's continuations, so we sample a transition (s,a,r,s') and form the target $y = r + \gamma \max_{a'} Q(s',a')$, then move $Q(s,a)$ toward y. If s' is terminal (in this system: a root COMMIT has ended the episode, so $A(s')$ is empty), the continuation value is defined to be zero and the target is $y = r$; the terminal flag is the structural condition of Section 4.2 (a COMMIT whose target is the root), not a learned predicate. Q-learning is **off-policy**: it learns the optimal Q from experience generated by any adequately-covering behavior policy (random, epsilon-greedy, search-guided). That convergence-to-optimal statement is a tabular theorem (every state-action pair visited infinitely often, decaying step sizes); combining function approximation, bootstrapping, and off-policy data is the classic "deadly triad," so in the approximate form it is a motivation, not a guarantee. The approximate form replaces the table with Q_θ (the adapter and heads of Section 5), trained to reduce $(Q_\theta(s,a) - y)^2$ using a replay buffer and a periodically-synced target network Q_{target}, the standard stabilizers for exactly this regime.

Returns. A trajectory interleaves states, actions, and rewards,

$$s_0, a_0, r_0,\ s_1, a_1, r_1,\ \ldots,\ s_T, a_T, r_T,$$

ending with the terminal reward r_T, which in this system is the verifier's verdict at the root COMMIT, or 0 if the step budget expires with no root commit (Section 6). The intermediate r_t are all 0 in the first version (Section 6). The return from step t is the discounted sum of rewards from t to the end,

$$G_t \;=\; r_t + \gamma\, r_{t+1} + \cdots + \gamma^{T-t} r_T \;=\; \sum_{k=t}^{T} \gamma^{k-t} r_k.$$

A one-step target ($y = r_t + \gamma \max_{a'} Q(s_{t+1}, a')$) bootstraps after a single transition and is fragile when reward is sparse and only terminal; regressing $Q(s_t, a_t)$ onto a longer n-step or Monte Carlo return G_t is a remedy that needs no search (Section 7.2). One caveat, developed in Section 7.2: the max inside the one-step target is what makes Q-learning off-policy, and a longer uncorrected return gives that property up in proportion to its length, because every reward beyond the first follows the behavior policy rather than the greedy one. MCTS (Section 7.4) is itself a search procedure that can supply even better targets, but it is optional. Methods that sum over s' (value iteration, expectimax) need an enumerable model; samplers (Q-learning, and MCTS, which is sampled search) need only to draw (s', r). This MDP is generative-only.

3. The structured state

3.1 The forest

The state is an immutable reasoning forest; each action returns a new state. Nodes are of a few kinds:

- **Goals and subgoals**: things to be solved. The original problem is the root goal; subgoals are optional decompositions.
- **Hypotheses**: tentative, unverified claims attached to a goal.
- **Facts**: claims with provenance `given`, `derived`, or `verified`.
- **Verifier-call records**: input, output, and success flag of each call to the symbolic oracle, retained as part of the state.

Edges record parent and child relationships and, optionally, a connective (Section 3.3). The verifier sits behind a pluggable protocol, so the core is domain-agnostic: a domain supplies a concrete verifier and a node vocabulary.

For example, solving $|x-3|+|x+2|=7$ by case analysis induces a forest like the following (node kinds in capitals, verbs on the edges):

```
g0  GOAL  solve |x-3| + |x+2| = 7                                  (root)
|
+- h0  HYPOTHESIS  square both sides to remove the absolute values
|  \- ABANDON h0                (walked away: degree-4 blowup; no verdict claimed)
|
|  SPLIT : exhaustive case analysis on the sign changes at x = -2 and x = 3
|
+- sg1  SUBGOAL  case x < -2 :   -(x-3) - (x+2) = 7
|  |
|  +- f1  FACT (derived)        1 - 2x = 7
|  +- h1  HYPOTHESIS            x = -3
|  +- VERIFY h1 -> f2  FACT (verified)   x = -3 solves it and x < -2
|  \- COMMIT sg1 (solution := h1)        x = -3
|
+- sg2  SUBGOAL  case -2 <= x < 3 :   -(x-3) + (x+2) = 7
|  |
|  +- f3  FACT (derived)        5 = 7      (contradiction)
|  +- VERIFY f3                 f3 promoted to (verified): the case has no solution
|  \- COMMIT sg2 (solution := f3)        no solution in this case
|
+- sg3  SUBGOAL  case x >= 3 :   (x-3) + (x+2) = 7
|  |
|  +- h2  HYPOTHESIS            x = 4
|  +- VERIFY h2 -> f4  FACT (verified)   x = 4 solves it and x >= 3
|  \- COMMIT sg3 (solution := h2)        x = 4
|
```

```
+- h3  HYPOTHESIS (root)        x in { -3, 4 }
\- COMMIT g0 (solution := h3)   final answer x = -3 or x = 4    (verifier-gated, terminal)
```

Three things this makes concrete. The verified-versus-tentative split is visible: $x = -3$ is a hypothesis until `VERIFY` promotes it, and the derived fact `f1` stays tentative because nothing ever verifies it. Every case closes with a `COMMIT`: `sg1` and `sg3` with solutions, and `sg2` with the verified outcome "no solution in this case." A case that provably has no solutions is a *solved* subgoal; its refutation is a result the parent consumes (the root hypothesis $x \in \{-3, 4\}$ is licensed by all three verdicts, including `sg2`'s). And `ABANDON` appears exactly once, on `h0`: the policy walked away from an unpromising line of attack, claiming no verdict. It is never used for cases resolved by contradiction. Only the root `COMMIT` is verifier-gated and terminal.

3.2 The one load-bearing distinction

The single distinction that must exist is verified facts versus tentative hypotheses. Without it the controller cannot tell "I proved this" from "I think this," and `VERIFY` (Section 4) loses its meaning. Precisely: the line is verifier-verified versus everything else. `given` facts sit on the trusted side by fiat (they restate the problem, not an LLM claim). A `derived` fact, despite the name, sits on the tentative side: it is an unverified LLM computation, and it is exactly the kind of node `VERIFY` exists to promote. Only provenance `verified` is on the proved side, and only a verifier call puts it there.

Predicates such as `solved` and `verified` are derived from the structure: information the policy and lens may read, never a control-flow constraint. The only hard gate in the system is the verifier on the committed root answer; everything else is scaffolding the policy may use or ignore.

3.3 Subgoals are optional scaffolding, not proof obligations

Decomposition exists to give the lens something to focus on, not to impose a control flow. The policy may split a goal, work some children, ignore others, abandon a branch, or commit the root directly from a single hypothesis. Nothing forces it to discharge every child of a conjunctive node. Subgoal-solving is instrumental, never required for reward: the LLM can produce a correct final answer even when not every subgoal was solved explicitly.

So the AND/OR connective on a decomposition edge is optional, non-gating metadata. Under final-answer verification (GSM8K, the MATH dataset), AND/OR aggregation is not load-bearing for correctness; it is at most a hint to the policy and a cue to the lens. It would become load-bearing only if we verified the completeness of the reasoning (exhaustive case analysis, or proof obligations in a system like Lean), a deliberately deferred and larger setting. Whether a typed connective with aggregation, and a checked exhaustiveness obligation, helps is an ablation, not a v1 commitment.

3.4 The math instantiation, and the Markov bet

The first target is verifiable math. The math state instantiates the core with `facts` (given, derived, verified), open and closed subgoals, `hypotheses`, and a `verifier_log`, all carrying stable identifiers so actions can reference nodes safely. The verifier is SymPy. The terminal check on a committed answer is one deterministic call: no learned judge, no reward model, no preference data.

The Markov bet, stated honestly: the state is a compressed approximation of the true reasoning history, since a lossy parser maps LLM text into nodes. When a learned value looks wrong, suspect the parser before the network.

4. The action space: verbs as lenses

4.1 An action is a lens

An action is `a = (verb, target)`, where `target` is an identifier into the state. The verb is the kind of move; the target is the node it operates on; the frozen LLM, where the verb calls for it, supplies free-form content. Each verb is fully specified by five fields:

- `avail(s, t)`: a state-conditioned legality mask.
- `schema(s, t)`: a JSON schema constraining the LLM output.
- `view(s, t)`: the lens get, the projection of s rendered into the LLM context.
- `apply(s, t, o)`: the lens put, the localized update to target t and its neighborhood given LLM output o.
- a reward, zero for almost every verb (Section 6).

The transition samples o from the frozen LLM conditioned on $\text{view}(s, t)$ plus a verb suffix, then sets $s' = \text{apply}(s, t, o)$. Since $P(s' \mid s, a)$ integrates over LLM continuations, the MDP is generative: we sample transitions, we do not enumerate them.

4.2 The verb set: six core, one optional

```
SPLIT            structure : expand a goal into children
FORM_HYPOTHESIS  propose   : add a tentative, unverified claim
COMPUTE_SUBEXPR  propose   : add a derived fact (a computation step)
VERIFY           assess    : call the symbolic oracle; sole intermediate-reward channel
COMMIT           resolve   : close a goal with an outcome (terminal at the root)
ABANDON          walk away : drop an unproductive branch; no outcome claimed
CRITIQUE         assess    : optional, reward-free LLM self-assessment
```

The set covers the five operation families that the reasoning-systems literature collapses to (Appendix A). The propose/expand family contributes three verbs: `SPLIT` for expansion, and `FORM_HYPOTHESIS` and `COMPUTE_SUBEXPR` for the two epistemic flavors of proposing. Each remaining family contributes one verb (`VERIFY`, `COMMIT`, `ABANDON`), with `CRITIQUE`, the critique/refine family's verb, optional: six core verbs plus one optional. Per-verb rationale:

- **`SPLIT`** folds conjunctive decomposition ("solve all children") and disjunctive or exhaustive case analysis ("cover the parent"): the same move (expand a goal into children), differing only in the AND/OR semantics of the children, which is recorded as optional non-gating metadata (Section 3.3).
- **`FORM_HYPOTHESIS`** and **`COMPUTE_SUBEXPR`** stay distinct because the role of the node they write differs (a free-standing conjecture versus a computation step), though both start on the tentative side of the Section 3.2 line, and `VERIFY`'s reward gate depends on that line: a derived fact must never be treated as verified without a verifier route.
- **`VERIFY`** invokes the symbolic oracle and, on success, promotes the target to a verified fact. The verifier is the sole reward *source* and reaches the agent through exactly two verbs: `VERIFY` is the only channel for intermediate reward (gated and capped, and zero-coefficient in the terminal-only first version, Section 6), and root `COMMIT` carries the terminal verdict. `VERIFY` is kept strictly distinct from `CRITIQUE`.

- `COMMIT` closes a goal with an outcome. Its target is always a goal: `avail(s, t)` requires `t` to be an open goal with at least one hypothesis or fact beneath it, and `schema(s, t)` constrains the LLM output `o` to designate exactly one existing node id under `t` as the outcome, so no free-form answer text enters at `COMMIT` and the committed answer's provenance is always a node the forest accounts for. The outcome is a solution promoted from a hypothesis, or a verified refutation ("provably no solution in this case"): a case resolved by contradiction is a *solved* subgoal and is `COMMIT`ted, never `ABANDON`ed, because its refutation is a result the parent needs (and it is what the lens renders as the closed subtree's verdict). `COMMIT` is recursive but terminal only at the root: the structural fact `target == root` is what makes a commit terminal and verifier-gated, so there is no separate "final answer" verb. Subgoal commits are reward-free bookkeeping.
- `ABANDON` walks away from an unproductive branch: no outcome is claimed, no verdict is recorded, and the retained history leaves the branch revisitable. It is not negative closure; a branch resolved by contradiction is closed with `COMMIT`. It is kept as an emittable verb, not just search control, because at inference budget $B = 0$ (Section 8) the deployed model runs no search and must be able to express "this branch is unproductive, move on."
- **No `SUMMARIZE` verb**: compressing a closed subtree to its statement and verdict is a deterministic rendering property of the lens (Section 4.3).
- `CRITIQUE` is an optional, reward-free LLM self-assessment of a hypothesis against its goal; whether it earns its tokens is an ablation.

4.3 The lens (view) and the write-back (apply)

The default `view`, when the target is a subgoal, is grounded-local: the problem statement, the ancestor path from root to target, and the target's own subtree (its facts, hypotheses, open children). Siblings and unrelated branches are hidden; verbs that need more override the default. Closed subtrees render compactly as their statement plus verdict, which is the lens property that replaces a `SUMMARIZE` verb.

The `apply` write-back is localized: the parsed output mutates the target and its neighborhood (add a hypothesis under the goal, add a derived fact, mark a node verified, record a critique), producing the next immutable state. Because both read and write are scoped to the target's neighborhood, acting on a subgoal is identical to stepping a smaller MDP rooted there. Locality, modularity, and recursion are the same property, derived from the action interface rather than asserted.

4.4 The factored action and why the target is promoted

If the controller chose only a verb and let the LLM sample the target, credit would blur: a value over verbs averages over every target the LLM might pick, so the agent would never learn which node to act on. Promoting the target into the chosen action fixes this. The policy factors as

$$\pi(\text{verb} \mid s) \cdot \pi(\text{target} \mid s, \text{verb}),$$

a categorical verb head and a pointer head over the live identifiers in the state. The effective action set is `verbs x valid-targets`, which grows with the tree, which is why the target side is a pointer over live identifiers, not a fixed-length vector.

Invalid pairs are masked before the policy normalizes. Legality is pair-level and defined by `avail`: pair `(verb, target)` is legal in s iff `avail(s, target)` admits the verb. The mask is type-aware

per verb: `COMMIT` points only at open goals, `VERIFY` at hypotheses and derived facts, `SPLIT` at open goals, and so on. Verb-level legality is *derived*, never separately specified: a verb is legal in s iff some live target admits it (the OR over targets), so the verb head masks exactly the verbs with no legal target and the pointer head never normalizes over an empty set. $A(s)$ is nonempty in every non-terminal state by construction: the root goal is open in every non-terminal state (only a root `COMMIT` is terminal) and `FORM_HYPOTHESIS` on an open goal is always legal, so episodes end only at a root `COMMIT` or the step-budget terminal of Section 6.

---

5. The controller network and the encoder ladder

5.1 Global value, local generation

One split organizes everything: the learned heads value on the global state, while the frozen LLM generates content from a local view. Whether an action helps depends on the whole problem, so the policy and Q function embed the entire forest; but generation is short and on-topic, so the LLM sees only the verb's scoped view. The full state remains the Markov state; the view is a projection for generation, never the state itself.

5.2 Encoder signature and heads

To score a variable, growing set of targets, the pointer head needs a feature vector per live node, not a single global vector. The encoder therefore emits a pair: a global readout, and one vector per node.

```
encoder(s) -> (g, {h_k})           g = global readout, h_k = per-node vector

g    -> adapter MLP -> z
z    -> verb head    -> pi(verb | s)            (K logits; verbs with no legal target masked, S
(z, h_k, verb) -> pointer head -> pi(target | s, verb)  (over live nodes, masked)
(z, h_k, verb) -> Q head       -> Q(s, verb, target)    (scalar per legal pair)
```

A target Q network (Section 2) supplies the bootstrap target. Q_{target} is a periodically-synced frozen snapshot of every trainable parameter on the Q path: the adapter and the Q head (the frozen LLM encoder is shared, never copied). The scope matters: the adapter trains too, and a snapshot of the Q head alone would still chase a moving adapter. The LLM is always frozen; only the adapter and the three heads ever train (together under five million parameters). A Bellman update on $(s, (\text{verb}, \text{target}), r, s')$ touches the scored pair and shared body; other actions see zero gradient on that example.

5.3 The encoder ladder (a pluggable axis)

Three encoders, in increasing structural fidelity; the heads and loss are unchanged across them, so the choice is an experiment, not a commitment.

1. **LLM marker-token hidden states (v1 default).** Serialize the forest in root-to-children (pre-order) order with marker tokens (`[PROBLEM] ... [NODE id=sg_7 type=subgoal] <statement> ... [STATE_END]`) and run one forward pass; a transformer yields a hidden vector at every position, so all reads come from that single pass. The v1 reading rule: each

node's vector h_k is the mean over the node's own statement tokens, from its marker through the end of its own statement, excluding descendant spans (under pre-order, a node's own span ends where its first child's marker begins); the global readout g is the hidden state at `[STATE_END]`, which attends over the entire serialization. Marker-position-only reading and pooling over the full subtree span are ablation arms, not the default. Caveat: causal attention means a node's own tokens see ancestors and earlier siblings but not descendants, so a parent's vector does not reflect its children's progress; the global readout carries that information, and option 3 removes the limitation entirely. Deployment note: reading hidden states at chosen positions requires a raw transformers-style forward pass with hidden-state output; throughput engines behind completion or embedding APIs (vLLM- and TGI-style servers) do not readily expose per-token hidden states. In v1 the encoder is the same frozen checkpoint as the generator, co-hosted in one process serving both roles. Each controller step therefore costs one full-state encoding forward (prefill only, no decoding) in addition to the generation pass over the lens view; both count toward Section 9's forward-pass and FLOP totals.

2. **Text-embedding endpoint.** A decoupled sentence embedder over the serialized state gives a clean separation from generation and is compatible with standard embedding endpoints. It returns one pooled vector, so per-node vectors require one call per node, which is the cost of the cleaner separation.

3. **Semantic-feature graph neural network (the principled upgrade).** Because the state is genuinely a graph, a graph neural network (GNN) over it is the natural encoder. The essential design point: a topology-only GNN knows parent/child and connective edges but nothing about the mathematics, so each node's input features are initialized from a text embedding of its statement; typed edges (parent/child, fact/hypothesis, AND/OR) carry the relations; message passing then yields per-node and pooled-global vectors that are both semantic and structural, and bidirectional, which removes the causal-visibility limitation of option 1. Because nodes are immutable, their text embeddings are computed once and cached; only the cheap GNN re-runs as the forest grows, which is what makes the per-decision cost of this rung far below any LLM forward pass.

---

6. Reward: the verifier-anchor invariant

The symbolic verifier is the only reward source. The literature supports this: DeepSeek-R1 rejects learned process-reward models (PRMs) on reward-hacking and ill-defined-step-boundary grounds, CRITIC shows self-critique without external grounding is unreliable, and the most reliable systems (AlphaProof, AlphaGeometry, rStar-Math, R1, Tulu-3) are hard-verifier-anchored.

- Most verbs return reward 0 (`SPLIT`, `FORM_HYPOTHESIS`, `COMPUTE_SUBEXPR`, subgoal `COMMIT`, `ABANDON`, `CRITIQUE`). Lens compression, being a rendering property of the view rather than an action (Section 4.3), carries no reward by construction.
- `VERIFY` carries the design's only intermediate-reward channel: a gated, capped positive reward on a genuinely new promotion to a verified fact (anti-farming gates: per-episode cumulative cap, dedup per target, reject tautologies). In v1 this channel's coefficient is 0: every intermediate reward, including `VERIFY`'s, is 0.
- Root `COMMIT` returns the terminal verdict, $+1$ correct, -1 incorrect.

The first version uses terminal-only reward:

$$r_T = \begin{cases} +1.0 & \text{verifier judges the committed answer correct (root COMMIT)} \\ 0.0 & \text{step budget } T_{\max} \text{ exhausted with no root commit} \\ -1.0 & \text{committed answer incorrect (root COMMIT)} \end{cases}$$

with all intermediate rewards 0. $T_{\max}$ is a maximum number of controller actions per episode; it bounds the trajectory, not the forest depth. Budget expiry is treated as a genuine terminal ($y = r = 0$), and the remaining budget is part of the state, serialized in the state header so the encoder sees it (the immutable forest already implicitly encodes elapsed steps, since every action leaves a record, but the explicit feature makes the Markov property manifest rather than inferred).

The gated, capped `VERIFY` channel is the pre-specified fallback, and the only sanctioned one: its magnitude, per-episode cap, and activation criterion (a pre-registered stall test, for example learned-controller accuracy at $B = 0$ failing to improve over K consecutive evaluation checkpoints) are fixed in the pre-registered experiment plan (Section 9.1) before any run enables it; they are not tuned mid-experiment. "More signal" always means more verifier-derived shaping, never a reward model and never an LLM-judge reward. The moment reward comes from a model's opinion rather than the verifier's check, the project has changed. Aesthetic signals (reasoning-quality classifiers, stylistic critics) may enter only as MCTS priors or pruning heuristics, never as Bellman targets.

---

7. Training: value-based RL as the spine

7.1 The core: off-policy value-based Q-learning

The core is the approximate Q-learning of Section 2 over sampled transitions (s, a, r, s') with $a = (\text{verb}, \text{target})$. The target is

$$y \;=\; r \;+\; \gamma \max_{(\text{verb}', \text{target}') \in A(s')} Q_{\text{target}}(s', \text{verb}', \text{target}'),$$

with the convention that the max term is zero when s' is terminal (root COMMIT, or budget expiry under Section 6's terminal convention), so $y = r$ on exactly the transitions that carry the verifier's verdict; the n-step and Monte Carlo returns of Section 7.2 use the same convention at the episode end. We update the Q head and shared body to reduce $(Q(s, \text{verb}, \text{target}) - y)^2$.

Roles: the heads act, Q criticizes. The factored verb and pointer heads (Section 4.4) are the acting policy at every stage and at deployment: sample verb from $\pi(\text{verb} \mid s)$, then target from $\pi(\text{target} \mid s, \text{verb})$, with explicit exploration (temperature, or epsilon-greedy over legal pairs). Q is the critic. The heads are refreshed toward the improved policy by distillation: the factored joint $\pi(\text{verb} \mid s)\, \pi(\text{target} \mid s, \text{verb})$ is trained by cross-entropy toward $\text{softmax}_{a \in A(s)}(Q(s, a)/\tau)$ over the legal-pair set (toward π_{MCTS} when search is on, Section 7.4). The distillation is gated and annealed: it begins only after Q's regression error on the warm-start data falls below a threshold (or after a fixed warm-up), so a cold Q never overwrites the behavior-cloned heads (Section 7.3). A learned Q remains the natural object: it folds the expensive expectation over LLM transitions into a cheap action value, so action selection at deployment samples no LLM transition to look ahead.

Off-policy is load-bearing: with the one-step max target above, the same replay buffer can be fed by random exploration, an LLM-controller (Section 7.3), or MCTS, with no change to the update. The qualifier matters: the longer returns of Section 7.2 weaken exactly this property, and Section 7.2 states the bias and the convention that manages it.

Replay rows and encode cost. Computing the max over $A(s')$ needs the per-node vectors and legality mask for s', which under the v1 encoder would cost one full LLM forward pass per replay row per update. Because the v1 encoder is frozen, this is avoided by construction: each replay row stores the raw states s and s' plus the encoder outputs $(g, \{h_k\})$ for both and the mask $A(s')$, computed once at collection time and never stale, so replay training touches no LLM forward pass at all (gradients flow only through the adapter and heads). The text-embedding rung caches the same way; the GNN rung is trained, so cached GNN outputs go stale and that rung re-encodes from the stored raw state (its frozen per-node text embeddings remain cacheable). Every replay row carries a (serialization-schema version, encoder id) tag; rows whose tags mismatch the current configuration are dropped or re-derived from the stored raw state, never silently consumed. This is load-bearing because the buffer is long-lived by design: rung-1 teacher trajectories (Section 7.3) are replayed at rung 3, potentially across encoder and serialization changes.

7.2 Sparse reward: n-step and Monte Carlo returns

A one-step bootstrap is fragile under terminal-only reward. The search-free remedy is to regress $Q(s_t, a_t)$ toward an n-step or full Monte Carlo return G_t from the verifier terminal (Section 2).

The price must be stated rather than hidden: every reward beyond the first inside such a return was produced by the behavior policy μ that generated the trajectory, so an uncorrected long return estimates Q^μ (take a_t, then follow μ), not $Q^\star$. Since $Q^\mu \leq Q^\star$ for every μ, returns from a strong behavior policy are tight, useful lower bounds on $Q^\star$, while returns from weak or random behavior are severely pessimistic. The design therefore commits to a convention per data source:

- **Near-on-policy data** (the controller's own recent behavior): uncorrected short-horizon n-step, $n \in [3, 5]$, the standard deep-Q convention; the bias is small because the behavior is close to greedy-by-Q.
- **Genuinely off-policy data** (teacher trajectories, stale replay): either cut the return at the first action that disagrees with the current greedy policy and bootstrap there (tree-backup / Retrace-style truncation), or use the full return deliberately as a warm-start labeled for what it is: regression toward Q^{teacher}, valuable precisely because the teacher is strong and Q^{teacher} is a tight lower bound on $Q^\star$.
- **Schedule**: long returns while teacher data dominates the buffer, shortening toward the one-step target as the controller's own behavior takes over, recovering the full off-policy property of Section 7.1 in the limit. Exact n and the switch-over are fixed in the pre-registered experiment plan (Section 9.1).

7.3 The three-rung controller-training ladder

Learning a policy over a variable, growing `(verb, target)` space under sparse reward is the highest-risk part of the design. A ladder defangs it by never asking the network to learn from scratch.

Two LLM roles are distinct. The *generator* is the small frozen LLM that produces content under the lens; it is the deployed model and the small-model thesis is about it. The *controller-teacher*

is an LLM used only at training time to pick `(verb, target)` decisions; it has no deployment footprint and may be much larger, even a frontier model.

Generator model class. The deployed generator is a small open-weights model, on the order of 1 to 4 billion parameters. Instruction tuning is permitted, and in practice needed, because every verb constrains the generator's output with a JSON schema (Section 4.1). What is excluded is reasoning-specific post-training: no o1/R1-style long-chain-of-thought RL and no R1-distilled checkpoints. The exclusion is load-bearing for the contrast of Section 1.2: with a reasoning-tuned generator, deliberation would happen inside each verb's generation as internal chain of thought rather than in the orchestrated state, and gains could no longer be attributed to the controller and the structure. (A pure base model driven by constrained decoding is the attribution-maximal variant, recorded as an ablation.) The exact checkpoint is delegated to the pre-registered experiment plan (Section 9.1); the class constraint (small, open-weights, instruction-tuned, non-reasoning) is a design commitment.

The ladder:

1. **LLM-as-controller (no training).** A controller-teacher picks the next `(verb, target)`, the small frozen generator fills in content under the lens, and the verifier checks. This validates the whole interface end to end before any network is trained, and its trajectories are the bootstrap data. To reduce the train-versus-deploy mismatch, the teacher orchestrates the small generator during collection, so trajectories reflect the content the deployed generator can actually produce: the teacher supplies decision quality, the small generator supplies realistic content. Stated honestly, this removes the generator-content mismatch but not the controller-induced state-visitation shift: the teacher's `(verb, target)` choices visit states the learned controller will not, so the rung-2 clone inherits the classic imitation covariate shift. Rung 3 mitigates this by consuming the controller's own off-policy behavior; if the cloned heads degrade off the teacher's distribution before rung 3 closes the gap, the named remedy is a DAgger-style pass (roll out the warm-started controller, query the teacher for `(verb, target)` labels on the states the controller actually visits, continue cloning on the aggregated data). The teacher remains a training-time labeler only; no reward is derived from it.

2. **Behavior-cloning warm-start.** Train the verb and pointer heads to imitate the teacher's chosen `(verb, target)` pairs, by cross-entropy on the factored heads over the encoder outputs $(g, \{h_k\})$. In the same rung, warm-start the critic: regress the Q head on Monte Carlo returns G_t computed offline from the teacher trajectories (the Section 7.2 machinery applied to the teacher buffer, labeled as regression toward Q^{teacher}). The heads start from "acts like the teacher" and the critic starts from "values like the teacher's returns" rather than from noise, which is what survives the sparse-reward cold start.

3. **RL fine-tune of the controller heads.** Act from the behavior-cloned heads with exploration (temperature or epsilon over legal pairs), update Q off-policy on the returns of Section 7.2 under its stated convention (trace-cut or warm-start-labeled full returns on teacher data, short-horizon n-step on the controller's own behavior), and distill the heads toward softmax-over-Q only on the gated, annealed schedule of Section 7.1, consuming both the teacher's trajectories and the controller's own behavior.

Per-stage roles, in one view:

Stage	Who acts	What trains
Rung 1 (collect)	controller-teacher	nothing (data only)
Rung 2 (warm-start)	offline	heads by BC; Q by MC regression on teacher returns
Rung 3 (RL)	cloned heads + exploration	Q (off-policy targets); heads (gated distillation)
Deploy $B = 0$	heads (greedy or low temperature)	nothing
Deploy $B > 0$	MCTS with heads as prior, Q at leaves	optionally distill from search

This is a distillation, but a different one from o1 or R1. Standard distillation compresses a big model's weights or chain-of-thought traces into a smaller model. Here the big model's weights are never touched and its chain of thought is never the product; we distill its policy over typed operations into a tiny external controller that drives a small frozen generator. Generation stays small and cheap; only the decision-making is borrowed, and only at training time.

7.4 MCTS as an additive add-on (not the spine)

MCTS is an optional accelerator that changes nothing about the core algorithm. Briefly: MCTS estimates action values by sampled search, building a tree of decision nodes (a state) and chance nodes (a state-action pair (s, a) holding a visit count $N(s, a)$ and a mean value). Each simulation selects actions by PUCT (predictor upper confidence bounds applied to trees),

$$Q(s, a) + c_{\text{puct}}\, P_0(a \mid s)\, \frac{\sqrt{N(s)}}{1 + N(s, a)},$$

where the prior P_0 is the learned policy (the factored heads of Section 7.1), $N(s) = \sum_b N(s, b)$ is the decision node's total visit count, and $c_{\text{puct}} > 0$ is an exploration constant (a pinned hyperparameter of the B3/B4 baselines, Section 9.1); expands a node; samples a transition and evaluates the leaf; and backs up the discounted return. The visit distribution $\pi_{\text{MCTS}}(a \mid s) = N(s, a)^{1/\tau_{\text{visit}}} / \sum_b N(s, b)^{1/\tau_{\text{visit}}}$ (at visit-count temperature τ_{visit}, distinct from the acting temperature τ) and the chance-node mean $\hat{Q}_{\text{MCTS}}(s, a)$ are the improved policy and value.

Because the core is off-policy, MCTS contributes in two purely additive ways: better behavior (search explores more intelligently than epsilon-greedy, adding higher-quality rows to the same buffer; those rows train with the one-step target or the chance-node regression below, both valid off-policy) and better targets ($\hat{Q}_{\text{MCTS}}$ beats a one-step bootstrap as a regression target for Q, and π_{MCTS} beats softmax-over-Q as a distillation target for the heads). Distillation from visit counts is exactly this add-on: the "MCTS-on" variant of the same losses, an ablation rather than a requirement. Keeping MCTS off-policy and additive is what makes "MCTS is optional" literally true. The on-policy policy-gradient alternative is recorded in Appendix B.

---

8. Inference-time compute knob

Once the controller is trained, the system trades latency for search via a budget $B \in \{0, 1, 2, ...\}$:

- $B = 0$: act directly from the verb and pointer heads (greedy or low temperature; Q available for tie-breaking). Per-step cost under the v1 encoder: one decode-free frozen-LLM prefill over the serialized state (producing $(g, \{h_k\})$), negligible head FLOPs, and one lens-scoped generation for verbs that produce content. The encoder pass is a full LLM forward pass and is counted as such in Section 9; per-decision cost far below any LLM forward pass is claimed only under the cached-embedding GNN encoder (Section 5.3, option 3).
- $B > 0$: run MCTS for B simulations from the current state, initialized from the learned heads as prior and Q at the leaves, and act on the improved visit distribution.
- Persistent search trees let a user "think harder" by continuing the search without restarting.

The budget is explicit and the compute is accounted for (Section 9).

---

9. Evaluation

The headline result is a compute-versus-accuracy frontier on verifiable-math benchmarks, escalating in difficulty: GSM8K, then MATH (evaluated on the standard MATH-500 subset), then AIME-style competition problems (for example AIME 2024/25, OlympiadBench) once the MATH frontier is non-trivial. The baseline grid is pre-registered in the experiment plan (Section 9.1) and not adjusted post hoc:

B0.	Deployed generator (same checkpoint), greedy decode	raw capability floor
B1.	Same generator, best-of-N, self-consistency majority vote over canonicalized final answers; N swept to trace its own compute-accuracy frontier	cheap deliberation
B1v.	Same generator, best-of-N, verifier-checked selection (gold-free substitute-back check per candidate); run only where the check needs no gold answer	strong deliberation
LCs.	Small generator as its own controller: handed the full serialized state, picks each (verb, target) zero-shot; no learned heads	deployable orchestration baseline
LCt.	Rung-1 teacher at inference: the controller-teacher picks (verb, target); the small generator fills content	orchestration ceiling
B2.	Learned controller, B=0 (no search)	fast mode
B3.	Learned controller, B=small MCTS	light search
B4.	Learned controller, B=large MCTS	heavy search
B5.	Larger reasoning model, greedy decode	capability ceiling

B0, B1, B1v, LCs, and LCt use the identical generator checkpoint as B2 to B4, so any gain over them is attributable to orchestration and training rather than to the generator's prior post-training; B5 is the only row using a different (larger, reasoning-tuned) model.

The compute axis is committed in advance and reported three ways together: total inference FLOPs per problem, total LLM forward passes per problem, and wall-clock at fixed hardware. Accounting

is itemized per row: (i) all frozen-LLM forward passes, prefill and decode, including encoder passes under the v1 encoder; (ii) controller head and adapter FLOPs (roughly 10^7 per decision, negligible); (iii) embedding-endpoint or GNN FLOPs where those encoders are used; (iv) verifier CPU time, reported separately. Controller forward passes count for every row: LCs adds one generator pass per action choice, LCt adds one teacher pass per action choice at the teacher's FLOPs. Comparisons are frontier-versus-frontier (B1's N is swept, not matched to a single row). Training-time teacher calls have no deployment footprint and are excluded from inference frontiers, reported once as training cost. The risk to guard against is a compute leak: reporting only the small model's FLOPs while spending many tokens elsewhere.

The honest win condition names its baseline. Against LCt the claim is distillation, not compute: the teacher's decision quality compressed into a roughly five-million-parameter controller that retains most of LCt's accuracy (beating LCt on compute is automatic and is not claimed as a result). Against LCs both axes are live: the learned controller should match or exceed LCs's accuracy while replacing a full prefill-and-decode LLM call per action choice with, under the v1 encoder, a decode-free prefill plus negligible head FLOPs (the saving is the decision-decode tokens and independence of decision quality from controller-LLM size; per-decision cost far below any LLM pass is claimed only under the GNN encoder). A sanity-check ablation (random Q head, uniform policy, fixed MCTS) confirms that training, not the bare scaffold, does the work.

Splits and contamination. Training and evaluation problems are disjoint by construction: rung-1 teacher trajectories, the rung-2 cloning set, and every problem that ever enters the replay buffer are drawn only from each benchmark's training split (GSM8K train; MATH train, excluding MATH-500); all reported numbers come from held-out problems no rung of the controller has seen. The frozen generator's pretraining has almost certainly included GSM8K and MATH; that contamination cannot be controlled, so it is handled two ways. First, every row runs the same frozen generator, so results are read as deltas over B0: pretraining contamination inflates the floor and the treatment rows alike, and the delta isolates what the controller and structure add. Second, the escalation includes at least one plausibly-uncontaminated set: problems released after the generator's training cutoff (the most recent AIME year) or synthetic problems generated and answer-checked by the verifier itself. Exact problem lists, split manifests, and seeds are fixed in the pre-registered experiment plan.

9.1 Pinned versus delegated quantities

Every load-bearing quantity is either pinned here or delegated to a single pre-registered experiment plan, `experiments/PLAN.md`, which this document commissions. That file is the registration referred to above: it is committed before the first training run and not adjusted post hoc.

Pinned at design level:

- Terminal reward $+1/0/-1$; v1 reward is terminal-only (`VERIFY` shaping off, Section 6).
- Discount $\gamma = 0.99$ (episodic, terminal-dominated reward; γ exists to weakly prefer shorter episodes).
- Controller: under five million trainable parameters (Section 5.2).
- Generator class: small open-weights, instruction-tuned, non-reasoning, roughly 1 to 4 billion parameters (Section 7.3); exact checkpoint named in the plan.
- Teacher default: a 70B-class open-weights model, chosen for reproducibility; a frontier API teacher is a sanctioned variant provided all rung-1 trajectories are archived, since an API model may change or disappear.

- Benchmarks and the baseline grid above.

Delegated to `experiments/PLAN.md` (each fixed there before any run):

- Episode step budget $T_{\max}$ (Section 6).
- Acting temperature τ (Sections 2, 7.1) and visit-count temperature τ_{visit} (Section 7.4); epsilon schedules.
- n for the n-step return, the trace-cut rule, and the return-length schedule (Section 7.2).
- `VERIFY` shaping, if ever activated: magnitude, per-episode cap, activation criterion (Section 6).
- Replay capacity, batch size, target-network sync period, optimizer, learning rates; the distillation gate and anneal schedule (Section 7.1).
- c_{puct} and concrete simulation budgets for B3 and B4 (for example $B_3 = 16$, $B_4 = 128$), fixed before the frontier is measured.

---

10. Non-goals and honesty discipline

- No reward model and no learned process-reward; the verifier is the sole reward source.
- No LLM-judge reward; `CRITIQUE` is reward-free information.
- No macro "solve it all" verb that bypasses per-step search and verification. A verb may write a localized multi-node chain (a step-by-step expansion under a subgoal), still atomic at the MDP level; what is excluded is a single verb that solves the whole problem.
- No regression of the classical correctness anchors: a small stochastic Gridworld with value iteration, expectimax, and MCTS triangulated against hand-computed values (in the style of Section 2's example) is a standing validation requirement, a non-regression gate the LLM system must never break.
- No search-first reframing. The central object is the agent (policy and Q) learning over the induced context MDP; MCTS is the policy-improvement operator and the compute knob.

Honesty boundary: this `(verb, target)` lens system is the designed method. Any presentation of this work marks explicitly which designed components (the pointer head, scoped lenses, the reduced verb set, optional `CRITIQUE`) are realized in the evaluated artifact and which are not; nothing still specified is claimed as built. Style: expanded sums before expectation notation, symbols defined at first use, hand-computable examples before general algorithms.

Appendix A. Prior-art grounding for the verb set

Across roughly thirty reasoning systems (Tree of Thoughts, Graph of Thoughts, RAP, LATS, ReST-MCTS*, rStar and rStar-Math, MCTSr, TS-LLM, Self-Refine, Reflexion, CRITIC, ReAct, Least-to-Most, Decomposed Prompting, Plan-and-Solve, ADaPT, Program-of-Thought and PAL, AlphaProof, AlphaGeometry, DeepSeek-R1, Tulu-3, STaR and V-STaR, Stream of Search and Guided Stream of Search (GSoS), Searchformer, Self-Backtracking, ReVISE, SCoRe, ASTRO), the operation vocabulary collapses to five families: propose/expand, verify/evaluate, critique/refine, commit/close, prune/backtrack. The verb set of Section 4 covers each family: propose/expand con-

tributes `SPLIT` (expand) plus `FORM_HYPOTHESIS` and `COMPUTE_SUBEXPR` (propose, split hypothesis-versus-fact), and the four remaining families contribute one verb each, with `CRITIQUE` optional.

Key data points behind the design:

- **Minimal verifier-anchored systems are tiny.** AlphaGeometry has two verbs (construct, deduce); AlphaProof is a single tactic space. This validates aggressive minimization.
- **Reward source is the load-bearing axis.** DeepSeek-R1 rejects learned PRMs; CRITIC shows external grounding is what makes critique reliable. Strong support for the verifier-only reward of Section 6.
- **Backtracking is search-control by default.** Every MCTS system puts it in selection; it becomes a learned, emittable action only in the internalized-search cluster (Self-Backtracking, ReVISE, SCoRe, ASTRO, GSoS), which is exactly the $B = 0$ deployment case that justifies `ABANDON` as a verb.
- **Closest structural analogue: TS-LLM** (PUCT MCTS, a learned value, and policy distillation, that is, generalized policy iteration). It differs principally in reward source: it trains an outcome reward model, whereas this project uses a symbolic verifier as the sole, untrained reward. That difference is the main novelty.
- **GSoS operation-level MDP** (each search operation is one discrete action) is the closest analogue to the `(verb, target)` action MDP and supports operation-level over token-level credit assignment.

Appendix B. The policy-gradient alternative

For completeness, the on-policy alternative to the value-based core of Section 7: sample trajectories under the factored policy, compute returns from the verifier terminal, and update by policy gradient with a learned value baseline (REINFORCE, PPO, or GRPO-style). The recent strong results in verifiable-reward reasoning are on this side, but at the token level over whole traces, which is the o1/R1 recipe this system contrasts against. Policy gradient is also on-policy, so it turns MCTS from an additive add-on into a fork (importance weighting, or a switch to distillation). For a small discrete action space with a natural learned Q, and to keep MCTS additive, the value-based core is preferred; policy gradient is the recorded alternative, not the default.